

2025-2026



**ESCOLA
SUPERIOR
DE TECNOLOGIA
E GESTÃO**

RELATÓRIO FINAL

MÉTODOS E TÉCNICAS DE SUPORTE AO DESENVOLVIMENTO DE
SOFTWARE - TRABALHO PRÁTICO

Grupo 7:

8180393 – Nuno Manuel Maia Moreira Costa Silva

8220337 – Hugo Ricardo Almeida Guimarães

8200594 – Diogo Miguel Ferreira Pereira

8190553 - Duarte José Ferreira Sampaio

Conteúdo

Índice de Figuras	3
1. Introdução.....	4
1.1. Contextualização	4
1.2. Objetivos do Trabalho.....	4
1.3. Estrutura do Relatório.....	4
2. Âmbito.....	5
2.1. Âmbito Funcional	5
Gestão da Estrutura Organizacional e Académica.....	5
Gestão do Ciclo de Vida do Aluno.....	5
Gestão de Colaboradores	5
Gestão de Avaliações e Notas	6
Autenticação e Autorização	6
2.2. Fora de Âmbito.....	6
3. Metodologia.....	7
4. Domínio Segundo o Domain Driven Design (DDD Estratégico).....	8
4.1. Linguagem Ubíqua.....	8
4.2. Contextos Limitados e Subdomínios.....	11
4.3. Endpoints.....	11
5. Infraestrutura	12
5.1. Kubernetes.....	12
5.2. Mecanismos de Comunicação	13
Comunicação Síncrona	13
Comunicação Assíncrona	14
6. Arquitetura dos Microserviços	15
6.1. Visão Geral.....	15
6.2. Descrição de Cada Microserviço	16
IAM.....	16
Candidaturas	17
Matrículas	18
Avaliações	19
Pautas.....	20
Horários	21
Gestão Académica.....	22
Planeamento de Avaliações	23
Notificações.....	24

7.	Processos de Negócio.....	25
	Lançamento de Classificações.....	25
	Candidatura Submetida	25
	Processo de Matrícula.....	26
8.	Diagramas de Estado.....	27
	Estados de uma Candidatura	27
9.	Bases de Dados.....	28
	Classificações	28
	Configuração de Avaliações.....	29
	Pautas.....	29
	Candidaturas.....	30
	Matrículas	30
	Gestão Académica	31
	Horários.....	32
	IAM	32
	Notificações.....	33
10.	Conclusão.....	34
	10.1. Reflexão Crítica dos Resultado	34
	10.2. GitLab e Jira	34
	10.3. Conclusões e Trabalho Futuro.....	35
11.	Anexos.....	36
	Referências	37

Índice de Figuras

Figura 1 - Contexto de Identidade e Acesso.....	8
Figura 2 - Contexto de Gestão de Alunos.....	9
Figura 3 - Contexto de Matrículas.....	9
Figura 4 - Contexto de Horários.....	9
Figura 5 - Contexto de Avaliações e Classificações.....	10
Figura 6 - Contexto de Relatórios e Análise.....	10
Figura 7 – Contexto de Comunicação e Notificações.....	10
Figura 8 - NAMESPACES KUBERNETES.....	12
Figura 9 - Exemplo de Ficheiros de Configuração.....	13
Figura 10 - Visão Geral dos microserviços.....	15
Figura 11 - Microserviço IAM.....	16
Figura 12 - Microserviço de Candidaturas.....	17
Figura 13 - Microserviço de Matrículas.....	18
Figura 14 - Microserviço de Avaliações.....	19
Figura 15 - microserviço de Pautas.....	20
Figura 16 - Microserviço de Horários.....	21
Figura 17 - Microserviço de Gestão Académica.....	22
Figura 18 - Microserviço de Planeamento de Avaliações.....	23
Figura 19 - Microserviço de Notificações.....	24
Figura 20 - Processo de lançamento de Classificações.....	25
Figura 21 - Processo de validação de Candidaturas.....	25
Figura 22 - Processo de Matrícula de um Aluno.....	26
Figura 23 - Diagrama de estados de uma Candidatura.....	27
Figura 24 - Base de dados do microserviço de Classificações.....	28
Figura 25 - base de dados do microserviço de configuração de avaliações.....	29
Figura 26 - Base de dados do microserviço de Pautas.....	29
Figura 27 - Base de dados do microserviço de candidaturas.....	30
Figura 28 - Base de dados do microserviço de Matrículas.....	30
Figura 29 - Base de dados do microserviço de Gestão Académica.....	31
Figura 30 - Base de dados do microserviço de Horários.....	32
Figura 31 - Base de dados do microserviço de IAM.....	32
Figura 32 - Base de dados do microserviço de Notificações.....	33
Figura 33 - Gráfico de Demonstração de divisão de trabalho.....	34

1. Introdução

1.1. Contextualização

Este trabalho foi realizado para o âmbito da disciplina de Métodos e Técnicas de Suporte ao Desenvolvimento de Software na Escola Superior de Tecnologia e Gestão do Instituto Politécnico do Porto, e tem como objetivo utilizar os conhecimentos adquiridos ao longo das aulas de modo a conceber uma aplicação inspirada no sistema Domus segundo o paradigma microserviços.

1.2. Objetivos do Trabalho

Este trabalho tem como principal objetivo o desenvolvimento de uma aplicação de gestão académica inspirada na plataforma Domus. De forma mais detalhada, pretende-se:

- Aplicar os conhecimentos adquiridos na unidade curricular para conceber uma solução assente no paradigma de microserviços;
- Definir a arquitetura técnica do sistema, incluindo a distribuição dos serviços e a forma de comunicação entre eles;
- Modelar os contextos funcionais segundo os princípios do Domain Driven Design;
- Especificar as bases de dados e a estrutura das APIs de cada microserviço;
- Implementar a solução e disponibilizá-la num ambiente de execução adequado, garantindo o correto funcionamento e a coerência entre os componentes.

1.3. Estrutura do Relatório

O presente relatório encontra-se organizado em vários capítulos que descrevem, desde a contextualização e análise do domínio, até à conceção da arquitetura baseada em microserviços, os modelos de dados definidos e a implementação da solução. Nos capítulos seguintes são apresentados as decisões tomadas, a metodologia seguida e os principais componentes que compõem a aplicação. Por fim, o relatório termina com uma reflexão global sobre o trabalho desenvolvido.

2. Âmbito

O presente projeto visa o desenvolvimento de um **Sistema Integrado de Gestão Académica (SIGA)**, uma plataforma centralizada e modular inspirada na abrangência funcional do sistema "Domus". O objetivo é modernizar e unificar a gestão de toda a informação e processos académicos para um grupo de instituições de ensino superior, desde a candidatura de um aluno até à conclusão do seu percurso.

O sistema será concebido para operar num modelo multi-instituição, onde uma administração central define as regras e estruturas globais, mas cada instituição (escola ou faculdade) mantém autonomia na gestão da sua própria oferta formativa e corpo docente.

2.1. Âmbito Funcional

Gestão da Estrutura Organizacional e Académica

Permite o registo e administração das universidades/escolas pertencentes ao grupo. Este domínio é responsável por configurar a base sobre a qual toda a atividade académica opera.

- **Estrutura Organizacional:** Registo de escolas, definição de políticas de avaliação globais e períodos académicos (anos letivos).
- **Oferta Formativa:** Gestão centralizada de cursos, planos de estudo e unidades curriculares (UCs).

Gestão do Ciclo de Vida do Aluno

Abrange todo o percurso do aluno na instituição, desde a sua entrada até à conclusão.

- **Candidaturas e Matrículas:** Gestão do processo de entrada de novos alunos e sua associação a um curso.
- **Inscrições:** Portal de self-service para o aluno se inscrever nas UCs do seu plano de estudos, com validação de regras (ECTS, precedências, etc.).
- **Gestão de Turmas:** Criação de turmas para as UCs, alocação de alunos e gestão de vagas.
- **Percurso Académico:** Portal para consulta do histórico de inscrições, pautas, estado académico (ativo, interrompido, concluído) e preparação para certificação.

Gestão de Colaboradores

Responsável pela gestão de docentes e outros tipos de funcionários, permitindo registar novas entradas, atualizar dados e gerir a sua alocação académica.

- Registo e manutenção dos dados de docentes e outros colaboradores.
- Associação de docentes às unidades curriculares que lecionam em cada ano letivo.

Gestão de Avaliações e Notas

O núcleo do processo de avaliação, desde a criação dos momentos de avaliação até à publicação dos resultados finais.

- Criação de avaliações (testes, trabalhos, exames) associadas a cada UC.
- Lançamento e atualização de notas pelos docentes para as suas turmas.
- Processo de validação, fecho e publicação de pautas pelos coordenadores de curso.
- Gestão do processo de revisão de notas solicitado pelos alunos.

Autenticação e Autorização

Assegura o controlo de acesso ao sistema com base em papéis e permissões. Garante que cada utilizador (administração central, gestor institucional, docente, aluno) apenas acede às funcionalidades e dados adequados ao seu perfil.

2.2. Fora de Âmbito

Para garantir o foco e a viabilidade, os seguintes domínios não serão incluídos na versão inicial do sistema:

- **Gestão financeira e administrativa:** Propinas, emolumentos e outros pagamentos.
- **Gestão de recursos humanos de âmbito contratual:** Folhas de pagamento, contratos e benefícios.
- **Funcionalidades de e-learning:** Gestão de conteúdos pedagógicos, submissão de trabalhos online ou plataformas de aulas virtuais.

3. Metodologia

Para desenvolver este projeto utilizou-se uma abordagem híbrida entre interativo e incremental, com o uso do SCRUM, e uma metodologia Waterfall. Esta mistura de metodologias foi utilizada pois nenhum dos membros do grupo sabia SpringBoot e Domain Driven Design, no entanto, como todos os membros já têm um conhecimento médio sobre desenvolvimento de software, como também as aulas de Domain Driven Design ocorreram no início do semestre, isso permitiu que, enquanto não eram lecionadas as aulas sobre o Spring Boot, a tecnologia a utilizar para a construção dos microserviços, se pudesse planejar de forma bastante pormenorizada a aplicação a desenvolver, desde os processos de negócio, passando pela criação de uma linguagem ubíqua, terminando com o planeamento pormenorizado de cada microserviço, assim, quando as aulas mais técnicas comesçassem, iríamos apenas ter que nos preocupar com a parte técnica, sendo isso uma característica de uma metodologia Waterfall.

No entanto, quando se passou para a fase de implementação, decidiu-se utilizar a metodologia SCRUM e realizar um sprint por semana, assim conseguia-se definir prazos, e visualizar se o projeto estava de acordo com o planeamento inicial, e dentro das datas estabelecidas internamente. Algo que também é importante mencionar, é que como ainda se estava a lecionar as ferramentas nas aulas, então nenhum dos elementos do grupo tinha proficiência com as tecnologias, logo, as reuniões constantes do SCRUM permitiram com que todos os microserviços permanecessem bastante semelhantes entre si, como também permitiu identificar dificuldades entre os elementos do grupo perante o trabalho a eles atribuído.

Depois da fase de implementação, realizou-se sprints somente para a testagem da aplicação, desde testes manuais, revisões de código, até os testes de integração. Sendo este ponto onde melhor se visualiza a abordagem híbrida adotada, pois normalmente, na metodologia SCRUM, uma sprint é conhecida como um “mini waterfall”, onde dentro do sprint existe a fase de planeamento, implementação e testagem, no entanto, na metodologia implementada, existiu sprints somente para testagem.

Para poder fazer a monitorização das tarefas utilizou-se o programa jira, onde a cada tarefa foi atribuído um conjunto de story points que refletem a importância e a complexidade dessa mesma tarefa.

4. Domínio Segundo o Domain Driven Design (DDD Estratégico)

Normalmente, quem programa um software, tende a focar-se demasiado na estrutura do código e acaba por esquecer algumas informações essenciais para o problema que o software a ser realizado visa resolver. Este problema piora, pois quem desenvolve o software utiliza uma linguagem técnica, onde somente pessoas da área do desenvolvimento de software entendem, fazendo com que as outras partes interessadas, que entendem o domínio do negócio e os problemas que o software a ser desenvolvido irá resolver, se afastem, criando uma barreira que poderá piorar o produto final [1].

Para resolver isso, surgiu o Domain Driven Design, introduzido pelo Eric Evans no livro “Domain-Driven Design”, onde, tenta-se ao máximo promover a colaboração entre todas as partes interessadas através de mecanismos que priorizam ao máximo o domínio, permitindo que o desenvolvimento ocorra de forma precisa e eficaz [1] [2].

No projeto desenvolvido utilizou-se uma abordagem DDD, sendo esta secção dedicada a explicar não só os mecanismos desta abordagem, como também como os mesmos foram utilizados no projeto.

4.1. Linguagem Ubíqua

Já que o principal objetivo da abordagem DDD é envolver todas as partes interessadas no projeto para o desenvolvimento do mesmo, logo é necessário com que todos falem “o mesmo idioma”, isto é, criar uma linguagem que seja compreensível para o domínio, isto permite com que todas as partes interessadas estejam envolvidas no projeto como trabalhem de forma eficiente [1]. Algo a destacar é que esta linguagem não deve ser somente utilizada no dia a dia, mas também deve estar presente em todo o código do projeto [1].

Para o projeto desenvolvido definiu-se a seguinte linguagem.

	Entidades	Descrição
1	Utilizador	Pessoa que acede ao sistema, podendo assumir diferentes papéis (ex.: estudante, docente, funcionário).
2	Função	Conjunto de permissões que determina o que um utilizador pode fazer no sistema (ex.: Docente, Estudante, Administrador).
3	Permissão	Ação específica que um utilizador tem autorização para executar (ex.: “Consultar Notas”, “Registar Presenças”).
4	Conta	Identidade digital associada a um utilizador, que inclui as suas credenciais de acesso.
5	Autenticação	Processo que verifica se o utilizador é quem afirma ser (ex.: login com palavra-passe ou autenticação institucional).
6	Autorização	Processo que define e valida o que o utilizador pode fazer, com base na sua função e permissões atribuídas.

FIGURA 1 - CONTEXTO DE IDENTIDADE E ACESSO

	🔖 Entidades	≡ Descrição
1	Aluno	Pessoa inscrita num ou mais programas oferecidos pela instituição.
2	Número de Matrícula (ID Aluno)	Identificador único atribuído a cada aluno.
3	Perfil de Aluno	Informações pessoais e académicas do aluno (morada, contactos dos encarregados de educação, dados de saúde, etc.).
4	Estado de Matrícula	Situação atual do aluno no processo de matrícula (Candidato, Matriculado, Desistiu, Concluído).

FIGURA 2 - CONTEXTO DE GESTÃO DE ALUNOS

	🔖 Entidades	≡ Descrição
1	Candidatura	Pedido de um aluno para ingressar num programa ou curso.
2	Matrícula	Confirmação de que o aluno foi admitido e inscrito num programa ou curso.
3	Vaga	Lugar disponível para um aluno numa turma ou curso.
4	Admissão	Processo de análise e decisão sobre a aceitação de uma candidatura.
5	Plano de Pagamento	Forma de pagamento das propinas ou taxas (parcelado, bolsa, isenção).
6	Confirmação de Matrícula	Etapa final que garante a vaga após o cumprimento dos requisitos (pagamento, documentos, vagas disponíveis).
7	Desistência	Quando um aluno abandona o curso antes da conclusão.

FIGURA 3 - CONTEXTO DE MATRÍCULAS

	🔖 Entidades	≡ Descrição
1	Agenda	Organização geral das aulas, docentes, salas e turnos.
2	Aula (Sessão de Aula)	Ocorrência específica de uma unidade curricular
3	Sala	Espaço físico ou virtual onde decorre a aula.
4	Horário	Combinação de dia e hora de início/fim.
5	Atribuição de Docente	Designação de um professor a uma aula ou turma.
6	Conflito de Horário	Sobreposição de aulas para o mesmo professor, aluno ou sala.

FIGURA 4 - CONTEXTO DE HORÁRIOS

	Entidades	Descrição
1	Avaliação	Instrumento de avaliação de desempenho (ex: Trabalhos, testes, exames)
2	Classificação (Nota)	Resultado atribuído à avaliação (numérico).
3	Livro de Notas	Conjunto de avaliações e classificações de uma unidade curricular.
4	Escala de Classificações	Regras que convertem pontuações em classificações ou médias.
5	Histórico Académico	Registo oficial de todas as classificações obtidas por um aluno.
6	Classificação Final	Resultado consolidado no final do período letivo.

FIGURA 5 - CONTEXTO DE AVALIAÇÕES E CLASSIFICAÇÕES

	Entidades	Descrição
1	Relatório	Visão consolidada de dados que apoia a tomada de decisão (ex.: assiduidade, desempenho académico).
2	Painel (Dashboard)	Conjunto visual de relatórios e indicadores.
3	Métrica	Medida quantificável (ex.: média de classificações, taxa de presenças).
4	Tendência	Variação de uma métrica ao longo do tempo.

FIGURA 6 - CONTEXTO DE RELATÓRIOS E ANÁLISE

	Entidades	Descrição
1	Mensagem	Informação enviada a um utilizador (e-mail, SMS, notificação push).
2	Modelo de Notificação	Mensagem pré-definida usada para eventos recorrentes.
3	Comunicado	Mensagem enviada a um grupo de utilizadores
4	Canal de Comunicação	Meio pelo qual a mensagem é enviada (portal, app, e-mail, SMS).
5	Gatilho de Evento	Acontecimento no sistema (ex.: "Aluno Matriculado") que gera uma notificação automática.

FIGURA 7 – CONTEXTO DE COMUNICAÇÃO E NOTIFICAÇÕES

4.2. Contextos Limitados e Subdomínios

Para lidar com domínios complexos e extensos, é útil dividir o domínio em modelos menores e separados, onde, ao invés de ter um único modelo abrangente, cria-se vários modelos menores, cada um deles com limites bem definidos e específicos sobre o que deve abranger [1] [2]. Já um contexto limitado é uma forma de estabelecer limites na aplicação, utilizando a linguagem ubíqua, de forma a evitar confusões com o resto do sistema [2] [3].

A abordagem DDD não costuma ser muito utilizado quando se trabalha com aplicações monolíticas, no entanto, o cenário já não é o mesmo quando se trabalha com microserviços, uma das principais razões, é que essa arquitetura encaixa perfeitamente com os conceitos de contextos limitados e subdomínios do Domain Driven Design, onde, cada microserviço é desenhado para corresponder a um contexto limitado específico, encapsulando um subdomínio bem definido, de forma a garantir alta coesão dentro do serviço, baixo acoplamento entre serviços e uma modelagem de negócio fiel às fronteiras identificadas no domínio [3]

Como a aplicação desenvolvida segue uma arquitetura em microserviços, logo cada contexto e subdomínio identificado foi convertido num microserviço. Sendo identificado os seguintes contextos:

- **Contexto de Candidatura** – Responsável pela submissão, validação e integração com a DGES;
- **Contexto de Matrículas** - Responsável pelo processo de matrícula; validação documental e criação de alunos no sistema académico;
- **Contexto de Gestão Académica** – Responsável por guardar informações relativas a cada aluno, docente, turma, unidade curricular e inscrições;
- **Contexto de Avaliações** – Responsável pela configuração de componentes; lançamento de notas; publicação das pautas e fluxos de revisão;
- **Contexto de Horários** – Responsável pela definição de slots de horário; agendamento de aulas; reservas de sala e conflitos;
- **Contexto de Identidade** – Responsável pelo registo; autenticação; gestão de papeis e permissões;
- **Contexto de Notificações** – Responsável pelo envio de emails e SMS.

4.3. Endpoints

Antes de falar sobre os endpoints, só a escolha da arquitetura REST para realizar as WebAPIs de cada microserviço já é algo que beneficia a aplicação do Domain-Driven Design, visto que esta arquitetura é bastante simples quando comparada a modelos SOAP e derivados [4], onde neste existem poucos comandos base (POST, PUT, GET, DELETE e PATCH) que são bastante simples e sobretudo autoexplicativos, que é uma característica importante para o DDD pois evita a criação de uma barreira técnica que poderá separar partes interessadas [1].

Mesmo utilizando uma arquitetura REST, a mesma pode não ser o bastante para a boa aplicação de uma abordagem DDD, visto que o nome dos endpoints deve ser autoexplicativo para quem

não é da área de tecnologia, que é algo que muitas vezes não é levado em consideração por quem programa, que já segue uma convenção para os nomes dos endpoints, que é autoexplicativo para quem programa, mas não para quem não trabalha com programação.

Como por exemplo, um programador caso queira mudar o estado de uma candidatura faz os seguintes endpoints: `PUT /api/v1/candidaturas/1?status="SUBMETED"`. Este endpoint é bastante autoexplicativo para quem programa, no entanto, para quem não programa irá perceber melhor se houver um endpoint explícito para cada estado e essa mudança estar explícita no nome, como por exemplo, invés do endpoint anterior, ter os seguintes: `PUT /api/v1/candidaturas/1/submeter` e `PUT /api/v1/candidaturas/1/anular-candidatura`.

Os endpoints desenvolvidos podem ser melhor visualizados na Secção 6.2.

5. Infraestrutura

5.1. Kubernetes

Todos os microserviços implementados no projeto estão totalmente configurados e operacionais para serem utilizados com o *minikube*, que é uma forma de executar o *kubernetes* localmente na máquina. Todas as configurações para tal encontram-se na pasta `app/k8s`.

Dentro da pasta de configuração do *kubernetes* encontra-se uma pasta com os ficheiros de configuração de cada microserviço, mas juntamente com essas pastas, existe uma denominada de namespaces que é fundamental para o bom funcionamento da aplicação. Um *namespace*, no *kubernetes*, é um espaço de nomes virtual que serve para isolar os recursos dentro de um mesmo cluster. No projeto desenvolvido, decidiu-se ter um *namespace* por serviço, de forma a existir um maior isolamento entre todos os microserviços, permitindo assim um maior controlo dos recursos, maior isolamento e controlo de recursos, mas acima de tudo, maior segurança

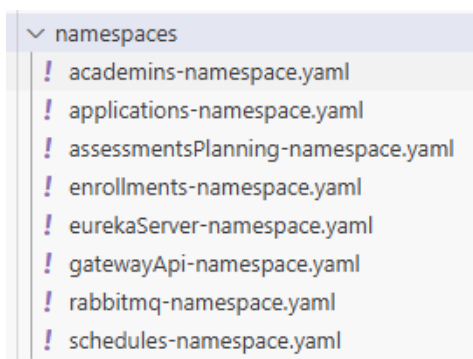


FIGURA 8 - NAMESPACES KUBERNETES

Para configurar cada microserviço, utilizou-se uma convenção, onde, cada microserviço contém um conjunto de ficheiros de configuração, sendo cada ficheiro responsável por algo diferente, permitindo assim, configurar mais facilmente cada microserviço. Cada pasta pode conter os seguintes ficheiros de configuração:

- ***-deployment.yaml** -> descrevem réplicas de pods, estratégia de rollout e imagens a usar
- ***-service.yaml** -> expõe pods internamente (ClusterIp) ou externamente (NodePort)

- ***-[BD]-stateful.yaml** -> definem a gestão de uma base de dados. Deve haver um ficheiro destes por base de dados utilizada no serviço
- ***-[BD].pvc.yaml** -> solicitam volumes persistentes para armazenar dados das bases de dados. Tal como o *stateful*, precisa de haver um destes por cada base de dados utilizada no serviço
- ***-config.yaml** -> são os *configMaps*, servem para manter as configurações não secretas
- ***-secret.yaml** -> guardam credenciais e dados sensíveis
- ***-init** -> é utilizado para inicializar a base de dados do microserviço (se for uma base de dados relacional precisa de ter a extensão .sql, caso contrário pode ser utilizado um .js)

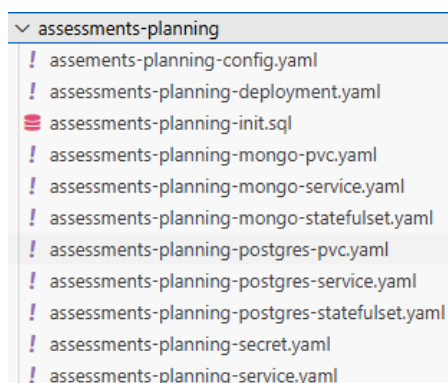


FIGURA 9 - EXEMPLO DE FICHEIROS DE CONFIGURAÇÃO

5.2. Mecanismos de Comunicação

Um dos maiores desafios quando se lida com uma aplicação dividida em diversos serviços independentes, é a forma como cada um comunica entre si, pois, embora eles sejam independentes, não é possível, nem lógico, fazer com que cada serviço fique dentro de uma caixa isolada sem nenhum tipo de comunicação com os outros ao redor.

Podem existir diversas razões para dois microserviços terem de comunicar entre si, e essa razão precisa de ser levada em conta para saber qual mecanismo de comunicação se irá utilizar. Portanto nesta parte irão ser explicados os diferentes tipos de comunicação entre microserviços, e quando os mesmo ocorrem.

Comunicação Síncrona

A comunicação síncrona acontece sempre que um microserviço necessita de informações provenientes de um microserviço para uma determinada ação, e não consegue finalizar a mesma sem obter uma resposta, como por exemplo: saber quais unidades curriculares o aluno está inscrito para poder exibir o horário.

Para além disso, este tipo de comunicação é essencial para poder assegurar a integridade referencial entre os microserviços, que é um dos principais desafios desta abordagem, sobretudo quando se lida com bases de dados relacionais, pois, como os registos estão fisicamente separados, então torna-se impossível definir determinados campos como chaves estrangeiras.

Para a comunicação síncrona entre microserviços, realiza-se pedidos usando o protocolo *HTTP* com o auxílio da biblioteca *WebFlux*, onde, caso se queira validar se um registo existe para poder fazer a garantia da integridade referencial, é realizado um pedido *HTTP* no começo do serviço através de uma classe do tipo *ApiClient*, que fica situada na pasta *clients*.

Comunicação Assíncrona

Para a comunicação assíncrona, que acontece sempre que um microserviço precisa de comunicar com outros, no entanto, não precisa de obter nenhum tipo de resposta, utilizou-se uma abordagem baseada em eventos e o sistema de mensagens *RabbitMQ*.

Este padrão é ideal para cenários onde a ação de um serviço deve desencadear processos em outros serviços, sem que o serviço que envia a mensagem fique bloqueado à espera de confirmações. Como por exemplo o envio de notificações, ou a criação de um registo de matrícula pendente sempre que o estado de uma candidatura é colocado como "aceite".

A implementação baseia-se no modelo de *Subscriber/Consumer*, onde um microserviço atua como publisher, publicando eventos (mensagens) para um *exchange* do *RabbitMQ*. E os microserviços interessados (*consumers*) inscrevem-se a esses eventos através de *queues* específicas, processando-as de forma independente e à sua própria velocidade.

6. Arquitetura dos Microserviços

Este capítulo é dedicado à descrição da arquitetura de microserviços do sistema, salientando que, apesar de nem todos os componentes terem sido implementados, existe um planeamento pormenorizado sobre cada um deles.

6.1. Visão Geral

Na Figura 10, é possível verificar que todos os microserviços, exceto as Notificações, comunicam de forma síncrona através da API Gateway, que se encarrega de organizar os pedidos e remeter ao microserviço requisitado.

Também é possível verificar que existe comunicação assíncrona entre Candidaturas e Matrículas, e entre Horários, Pautas, Classificações, Matrículas e Notificações. Estas comunicações assíncronas são possíveis através do RabbitMQ.

Como referido no canto superior direito, a entidade referencial entre os microserviços está assegurada, mas não foi representada para simplificar o diagrama.

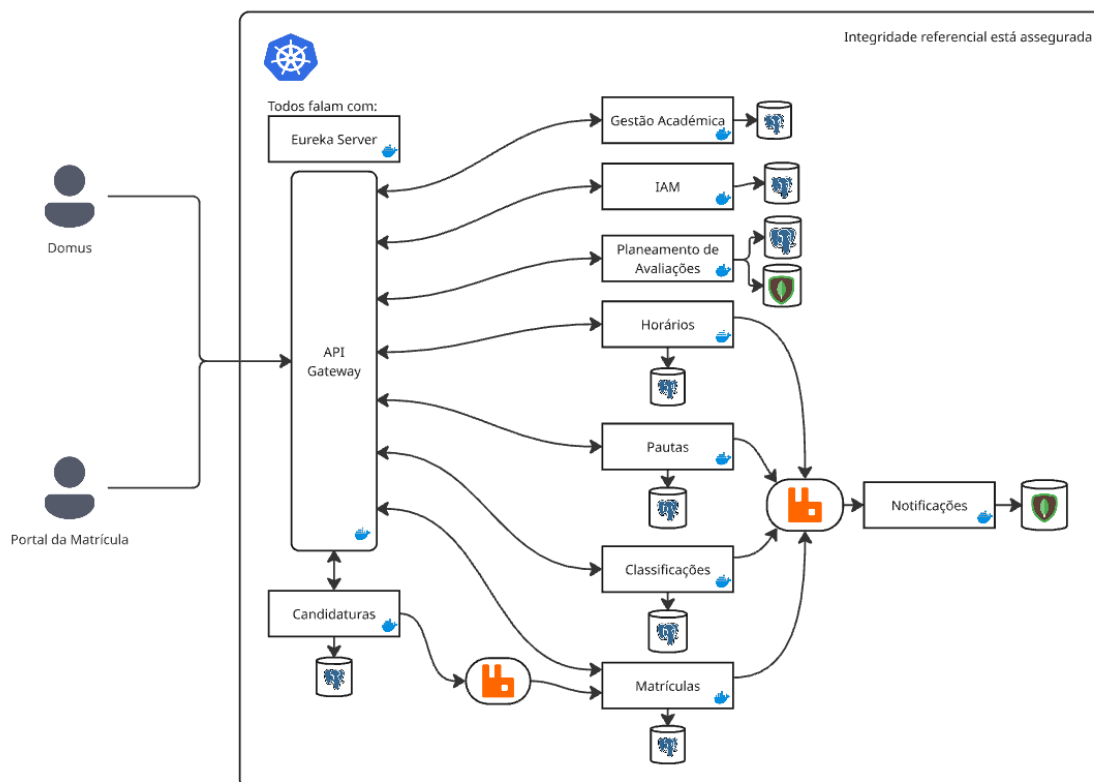


FIGURA 10 - VISÃO GERAL DOS MICROSERVIÇOS

6.2. Descrição de Cada Microserviço

IAM

Este é o microserviço responsável pela autenticação dos utilizadores e pela gestão de acesso às outras funcionalidades do sistema, restringindo o acesso de partes do sistema apenas a pessoas que contêm as devidas permissões.

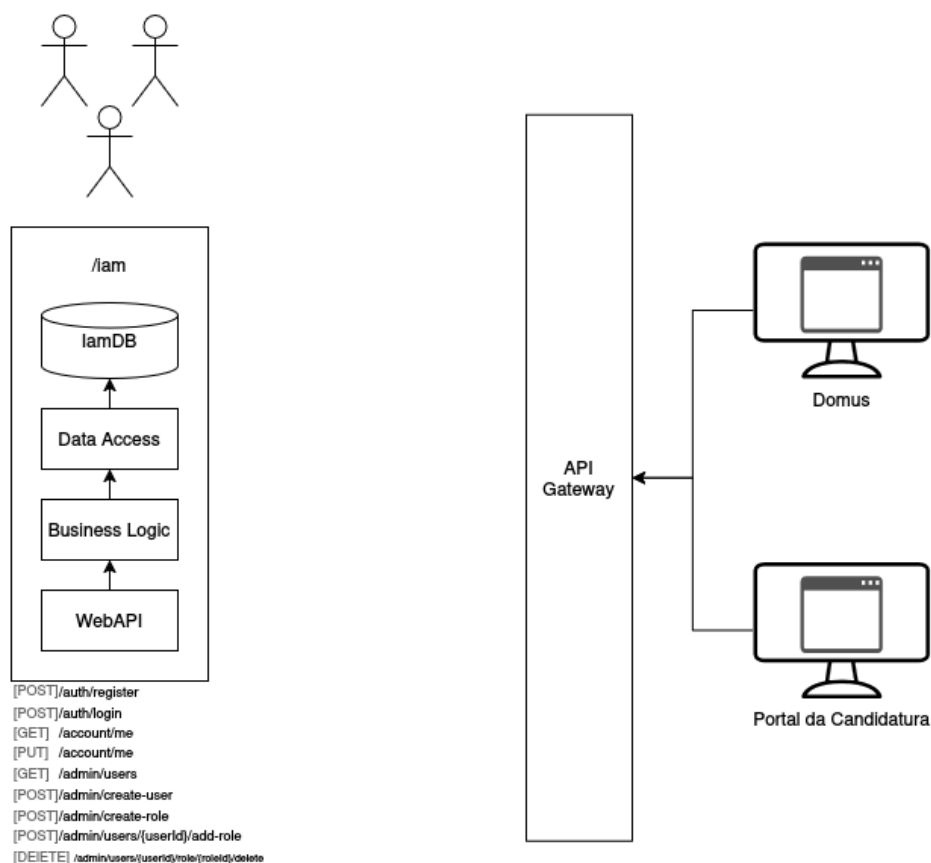


FIGURA 11 - MICROSERVIÇO IAM

Candidaturas

Este serviço é responsável por gerir os concursos especiais de acesso (M23, CTeSP, Mestrados, etc.) realizados tanto diretamente pela instituição como pela DGES.

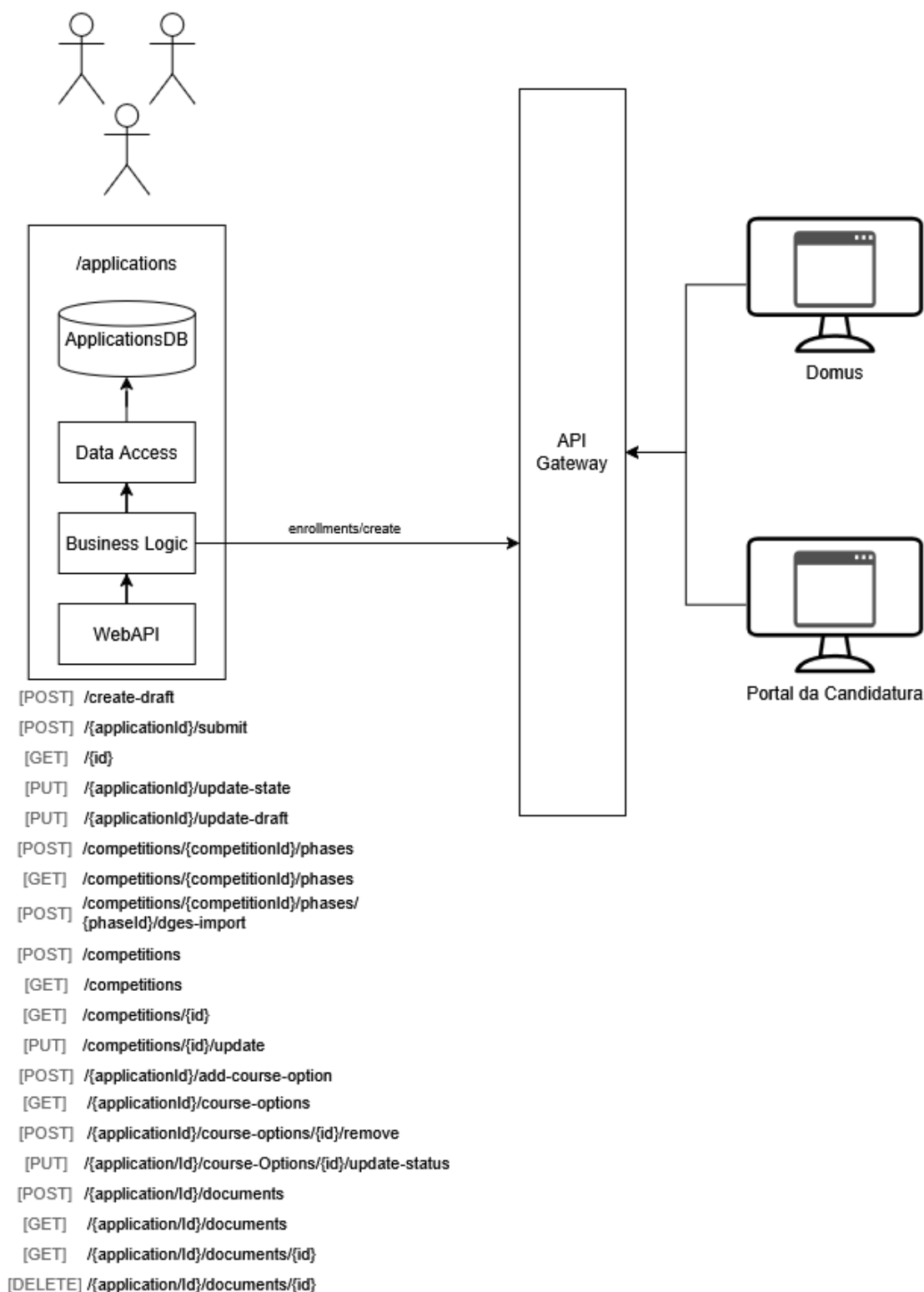


FIGURA 12 - MICROSERVIÇO DE CANDIDATURAS

Matrículas

Este serviço trata do processo final de admissão dos alunos, independentemente da origem dos mesmos (DGES ou concursos locais). Gere o processo de matrícula, incluindo a validação da documentação necessária. Resumidamente, é o registo oficial do aluno na instituição e a integração com outros sistemas académicos.

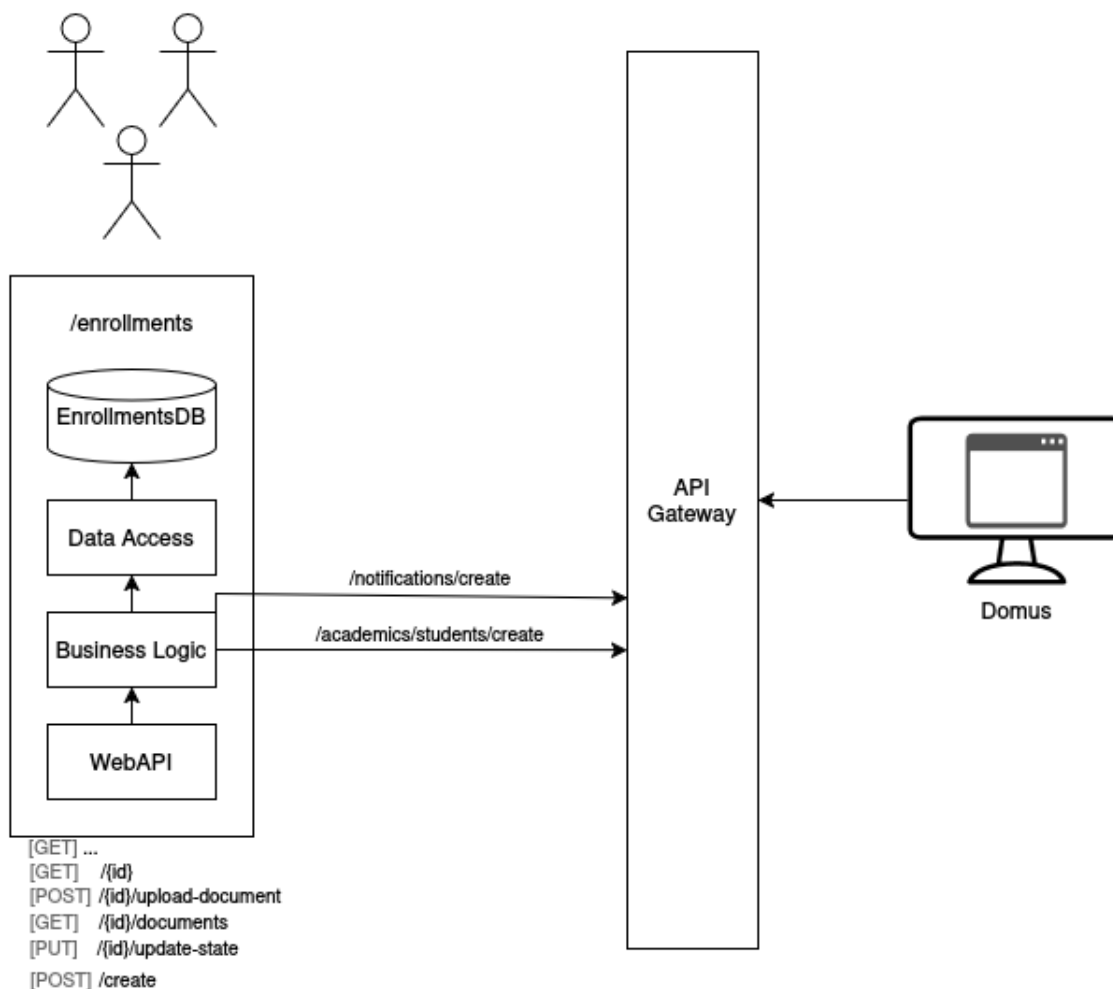


FIGURA 13 - MICROSERVIÇO DE MATRÍCULAS

Avaliações

Este serviço gere o processo de lançamento e gestão das notas individuais dos alunos, permitindo aos docentes registarem as classificações para cada momento de avaliação, mantendo um histórico detalhado de alterações de notas, para além de fazer o cálculo das médias por componente de avaliação. Funciona como o repositório principal das notas brutas e das alterações das mesmas ao longo do tempo.

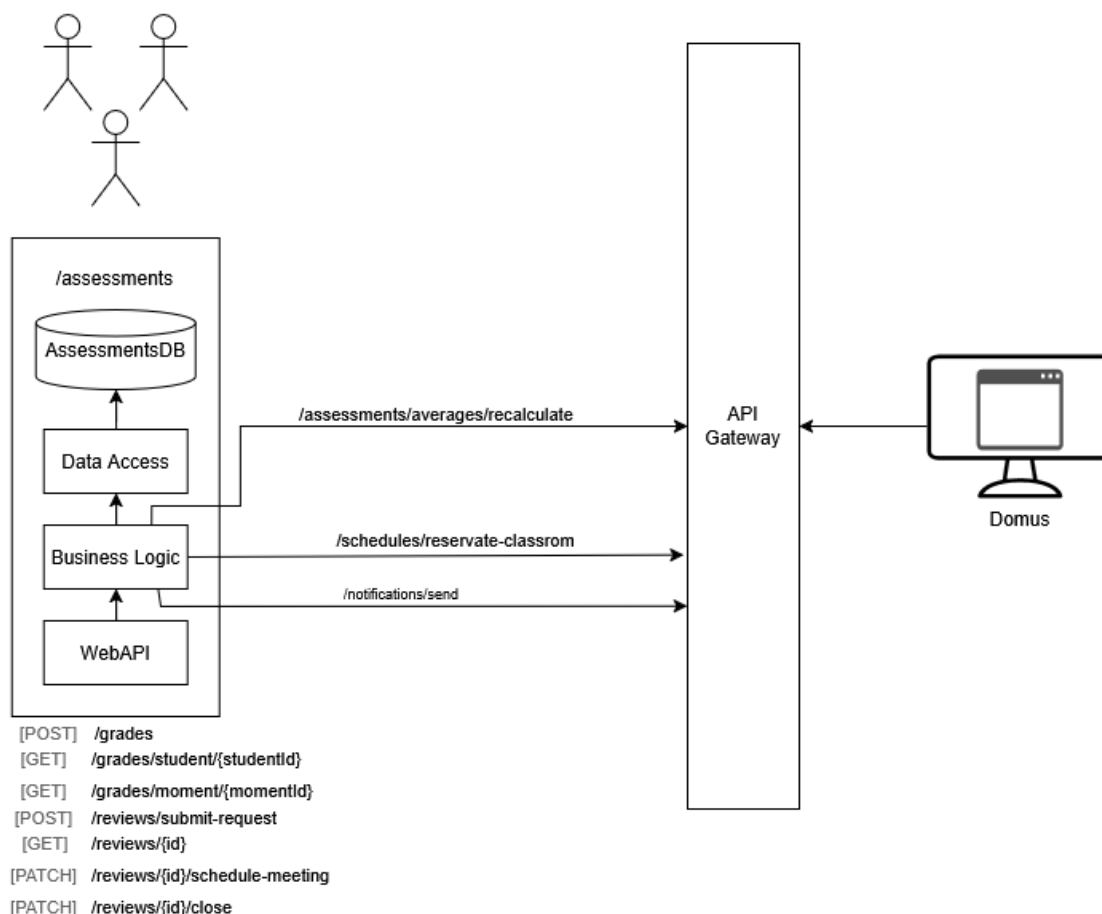


FIGURA 14 - MICROSERVIÇO DE AVALIAÇÕES

Pautas

Este serviço corresponde à fase final do processo de avaliação, sendo o responsável pela geração, validação e publicação dos resultados. Gere os diferentes tipos de pautas (normal, recurso, especial e melhoria), controla o fluxo de aprovação através de assinaturas de diversos intervenientes (docentes, coordenadores e diretor), e mantém o registo oficial do estado de aprovação dos alunos. É o componente que oficializa os resultados académicos.

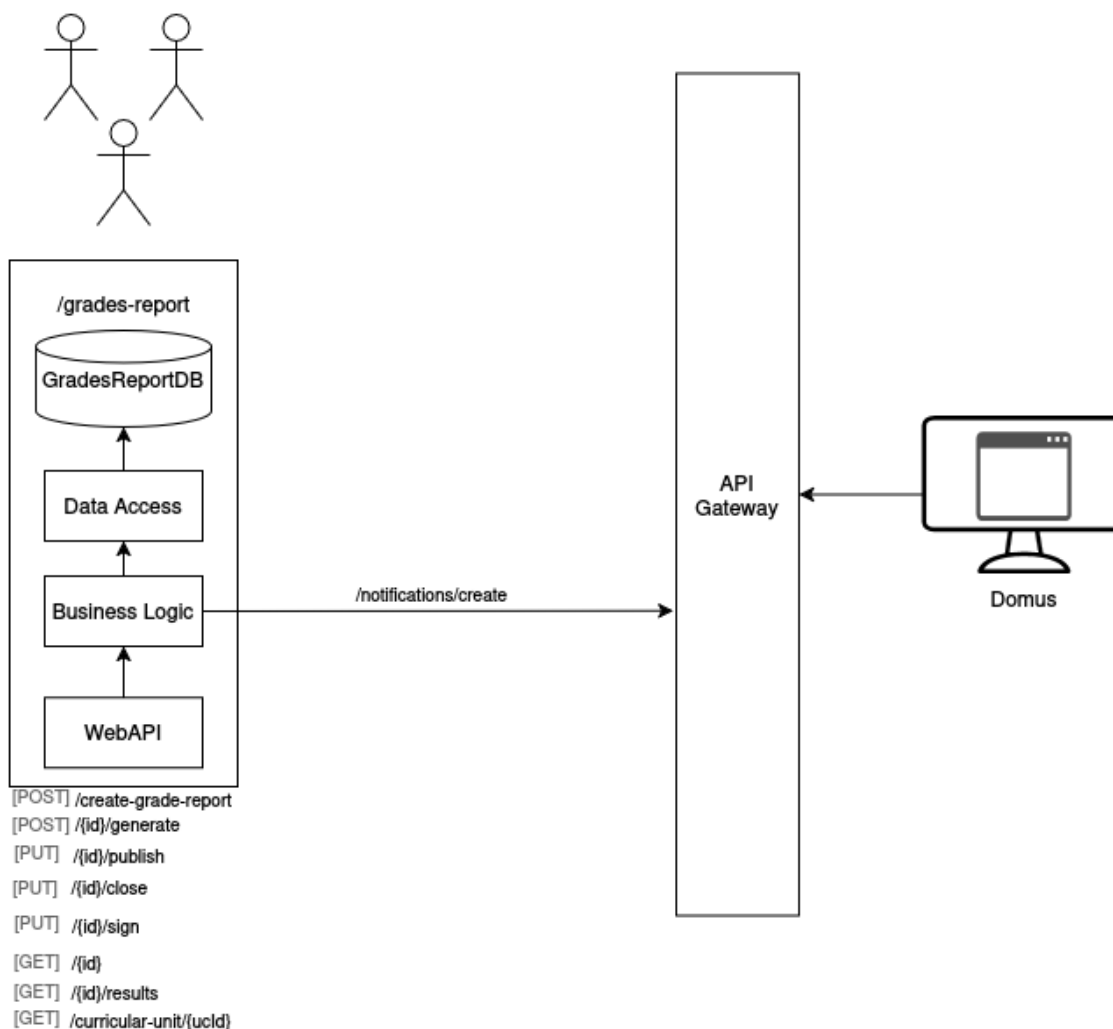


FIGURA 15 - MICROSERVIÇO DE PAUTAS

Horários

Este serviço está encarregue da gestão e organização temporal das atividades letivas, permitindo com que sejam criadas salas e slots de horário, permitindo assim com que as atividades letivas consigam ser distribuídas de forma espacial e temporal. Este serviço também permite a criação de exceções ao horário habitual (feriados ou alterações pontuais).

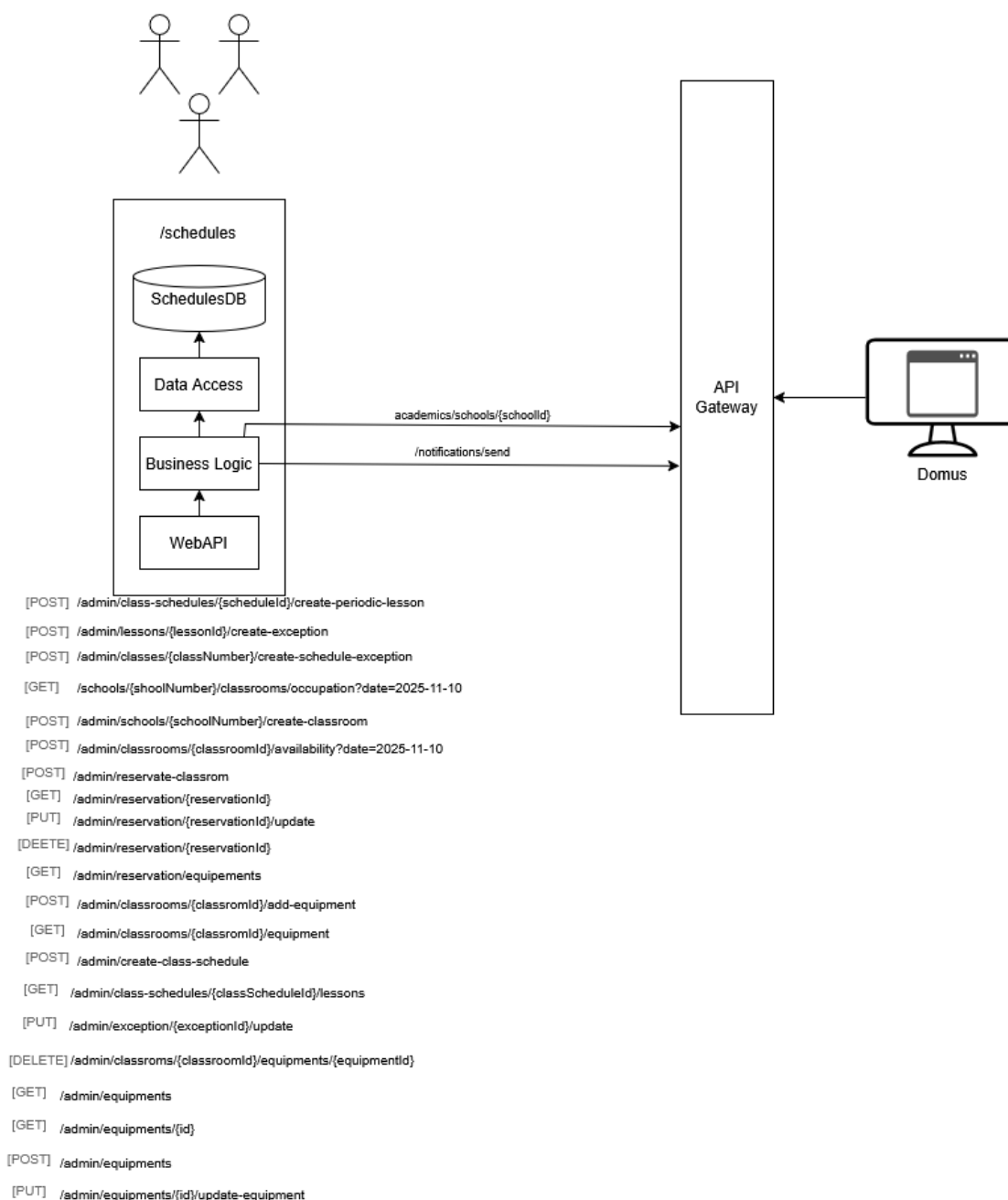


FIGURA 16 - MICROSERVIÇO DE HORÁRIOS

Gestão Académica

Este microserviço atua como o núcleo do sistema, pois é o serviço que fornece a estrutura fundamentas das instituições, cursos, unidades curriculares e turmas, para além de controlas as inscrições dos diferentes alunos entre as diferentes disciplinas da instituição onde estão matriculados

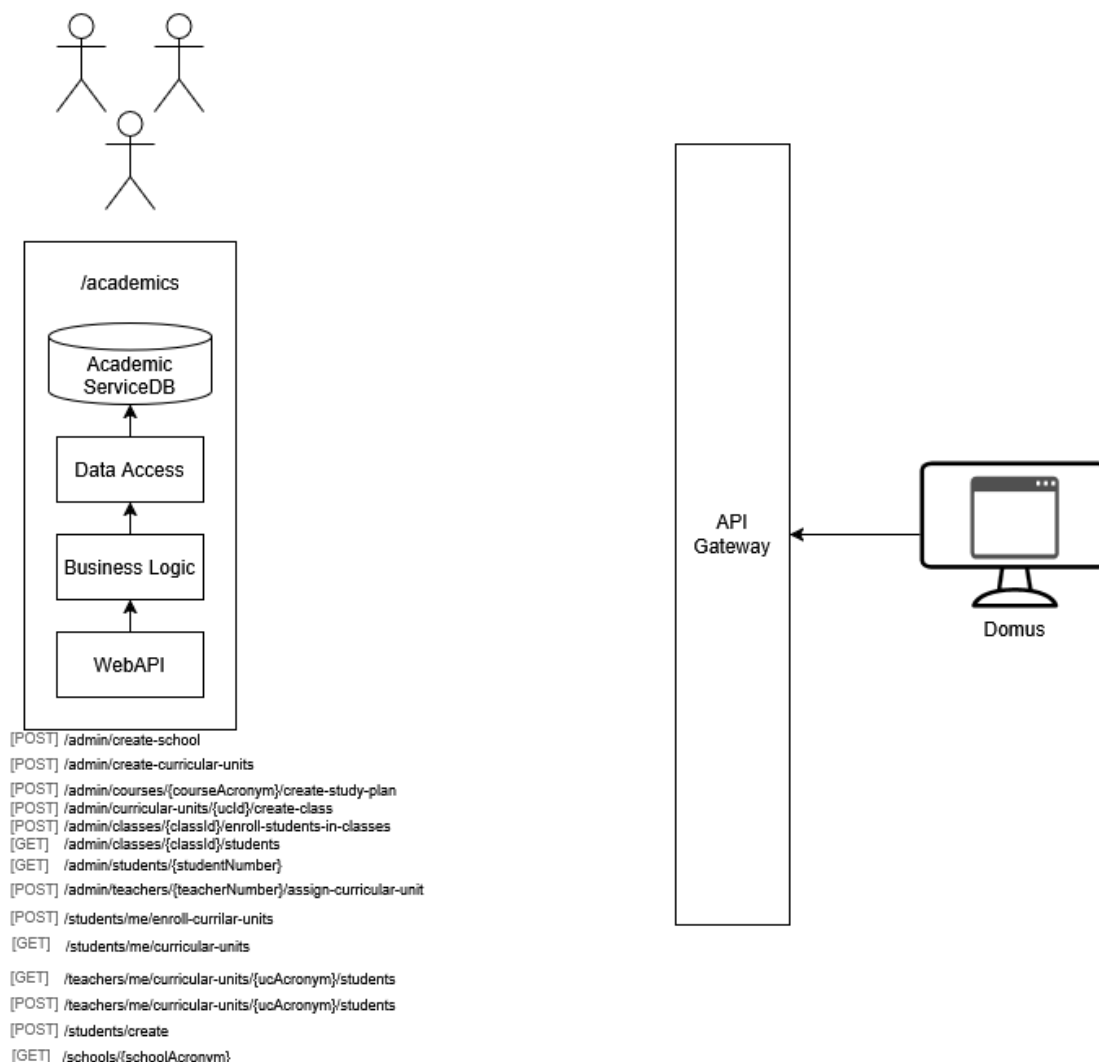


FIGURA 17 - MICROSERVIÇO DE GESTÃO ACADÉMICA

Planeamento de Avaliações

Este microserviço é responsável pela definição da estrutura base das unidades curriculares. Gere as escalas de classificação, define os componentes de avaliação (frequências, exames e trabalhos) e os respetivos pesos para a nota final, para além de controlar o agendamento dos momentos específicos de avaliação.

Resumidamente, é a parte de planeamento e configuração do processo de avaliação.

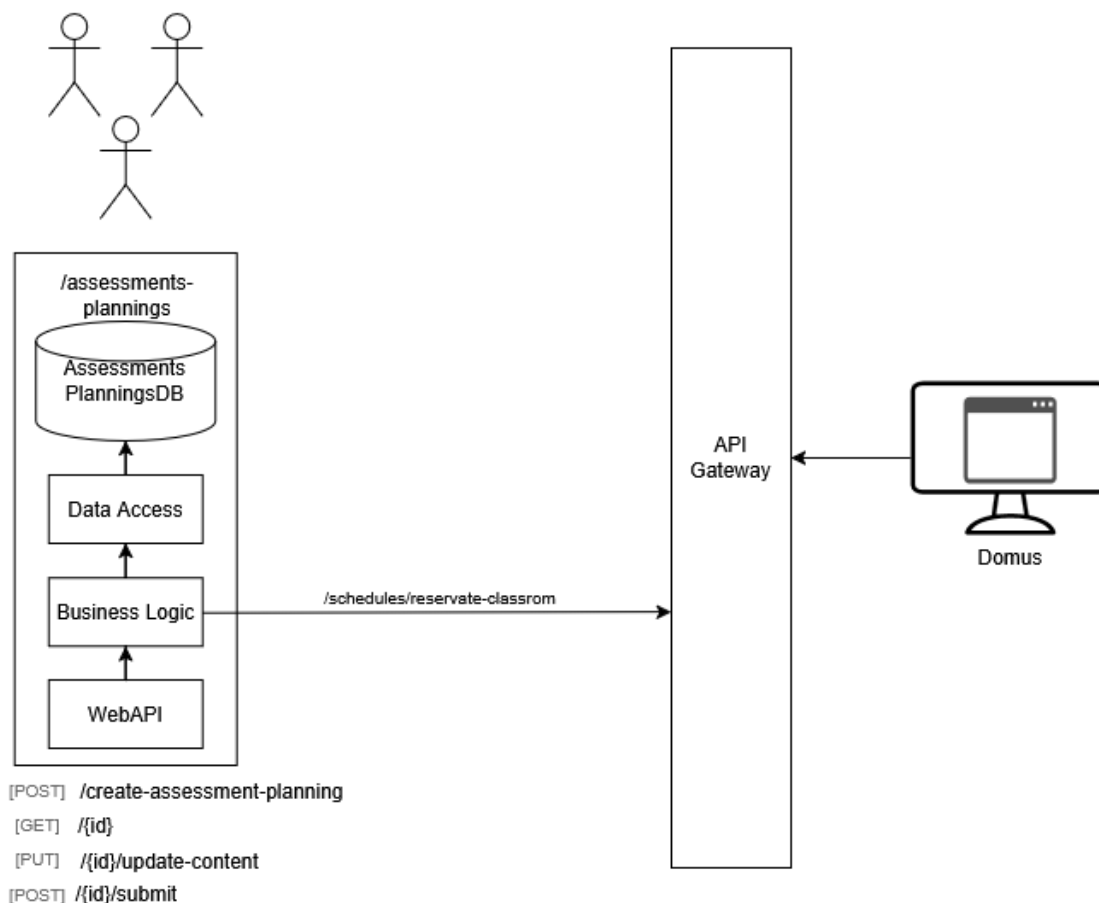


FIGURA 18 - MICROSERVIÇO DE PLANEAMENTO DE AVALIAÇÕES

Notificações

Este serviço é responsável pela garantia de uma comunicação eficiente entre todos os intervenientes, enviando alertas e informações relevantes para os alunos e docentes.

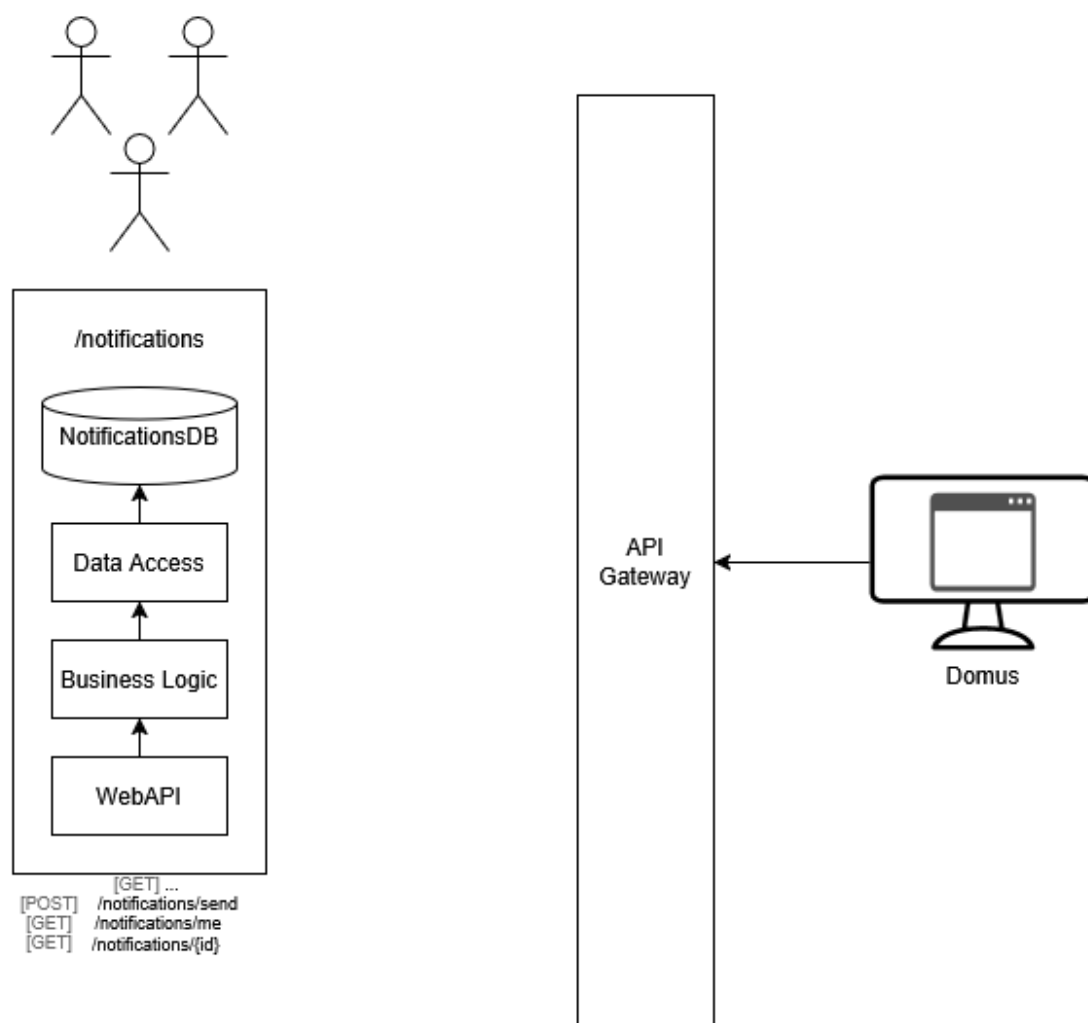


FIGURA 19 - MICROSERVIÇO DE NOTIFICAÇÕES

7. Processos de Negócio

Nesta secção, estão representados vários processos de negócio, modelados com recurso a BPMN (Business Process Model and Notation). Estes diagramas descrevem a sequência de ações, decisões e interações necessárias para completar uma tarefa de negócio específica, mostrando o fluxo de trabalho de ponta a ponta.

Lançamento de Classificações

Este processo demonstra o fluxo de trabalho que começa após a conclusão de avaliação e o posterior lançamento das classificações de uma Unidade Curricular. Aqui demonstra a possibilidade de marcação de uma reunião de revisão de nota e alteração desta, caso seja justificado.

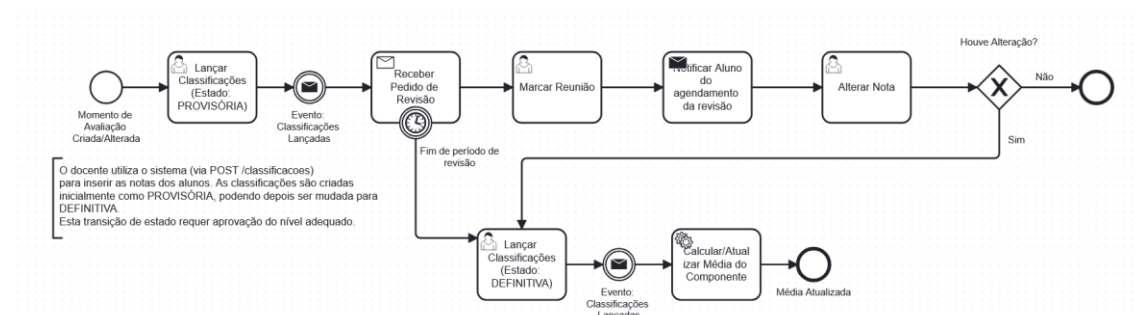


FIGURA 20 - PROCESSO DE LANÇAMENTO DE CLASSIFICAÇÕES

Candidatura Submetida

Este processo demonstra o fluxo de trabalho da validação de uma candidatura, após esta ter sido submetida. É possível visualizar o processo de validação de candidatura, a aprovação da candidatura e também a rejeição desta.

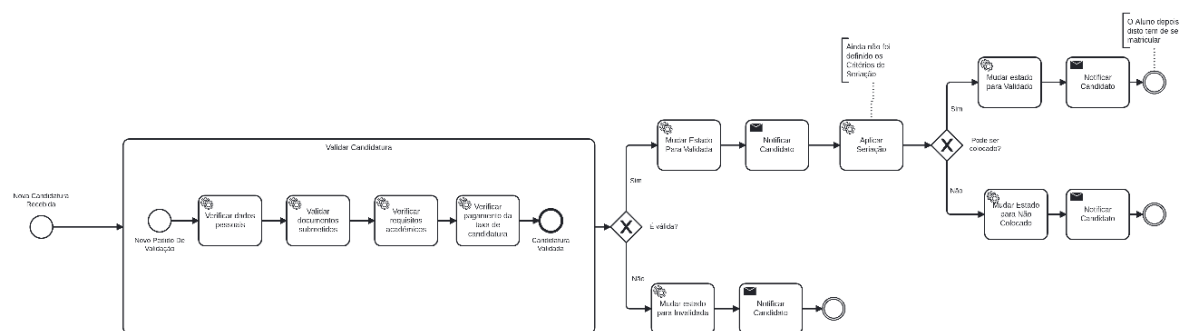


FIGURA 21 - PROCESSO DE VALIDAÇÃO DE CANDIDATURAS

Processo de Matrícula

Este processo demonstra o fluxo de trabalho da Matrícula. Ao efetuar a matrícula, o aluno submete documentos, que serão validados pelos Serviços Académicos da instituição. Caso não sejam válidos, a matrícula é rejeitada/invalidada. A gestão de pagamento, apesar de representada, encontra-se fora de âmbito.

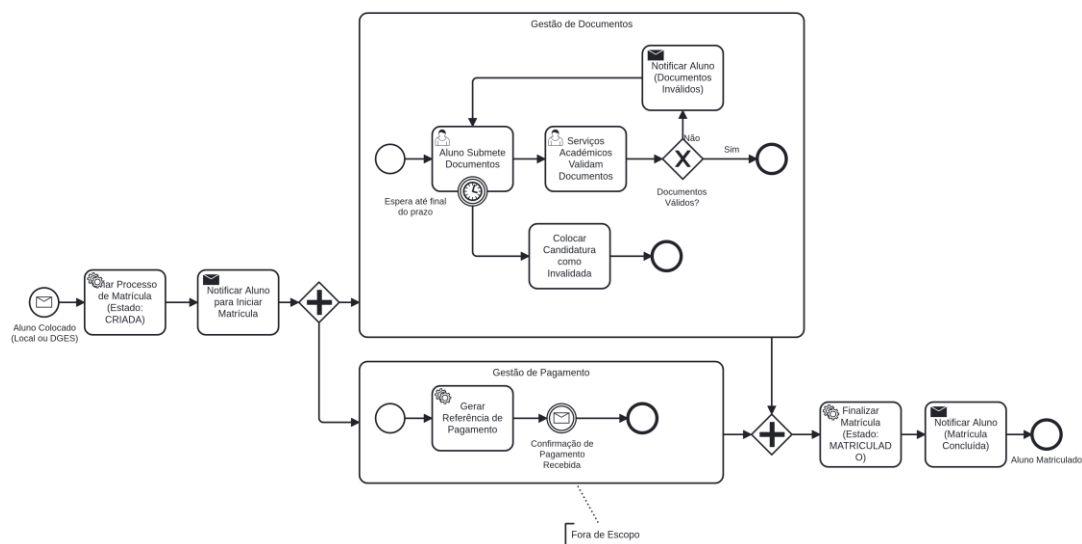


FIGURA 22 - PROCESSO DE MATRÍCULA DE UM ALUNO

8. Diagramas de Estado

Um Diagrama de Estados é um diagrama comportamental em UML (Unified Modeling Language). Este diagrama representa o ciclo de vida de um único objeto, sistema ou componente, mostrando as diferentes fases (estados) pelas quais ele pode passar e os eventos que causam a mudança de um estado para o outro.

Estados de uma Candidatura

Este diagrama de estados modela o ciclo de vida completo da entidade Candidatura, desde a sua criação até à decisão final. O diagrama define de forma clara todas as fases (estados) pelas quais a candidatura pode progredir.

O processo inicia-se no estado de Rascunho, seguido por Submetida e passa por avaliação de um administrador, que pode aprovar ou rejeitar a candidatura, dependendo do estado dos documentos fornecidos. O processo culmina nos estados terminais, sendo estes Invalidada, Não Colocado e Matriculado, fornecendo visibilidade total sobre o progresso e o estado atual de qualquer candidatura no sistema.

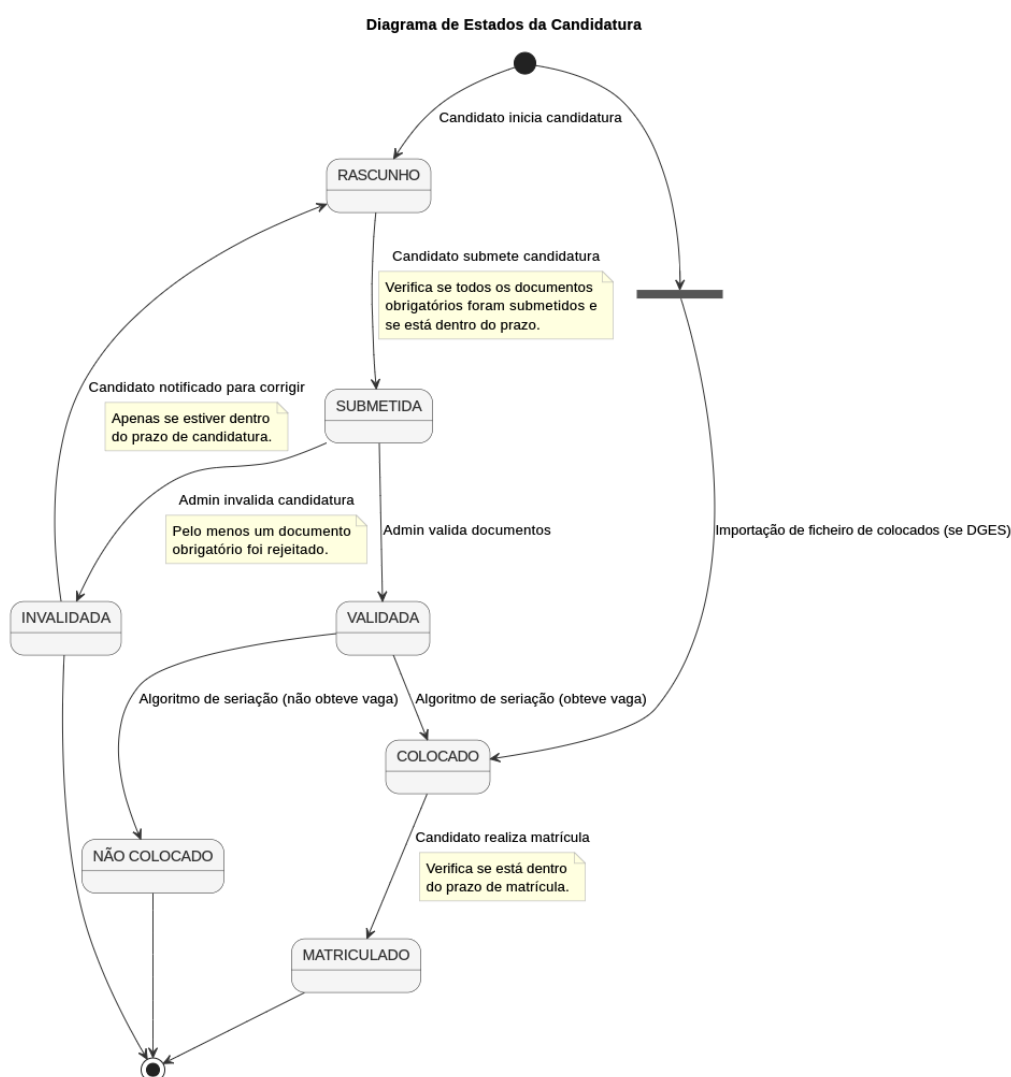


FIGURA 23 - DIAGRAMA DE ESTADOS DE UMA CANDIDATURA

9. Bases de Dados

Este sistema foi concebido sob uma arquitetura de microserviços, exigindo uma abordagem de design de dados descentralizada. A base de dados principal é logicamente separada, sendo cada esquema (ou base de dados dedicada) propriedade exclusiva de cada microserviço, garantindo o isolamento de dados e a autonomia de desenvolvimento.

Esta secção pretende expor a base de dados pensada para cada microserviço.

Classificações

Esta representação demonstra a base de dados da entidade “Classificações”, que será usada na atribuição de notas dos alunos, por parte de um dado docente, com a possibilidade de existir revisão dessa nota.

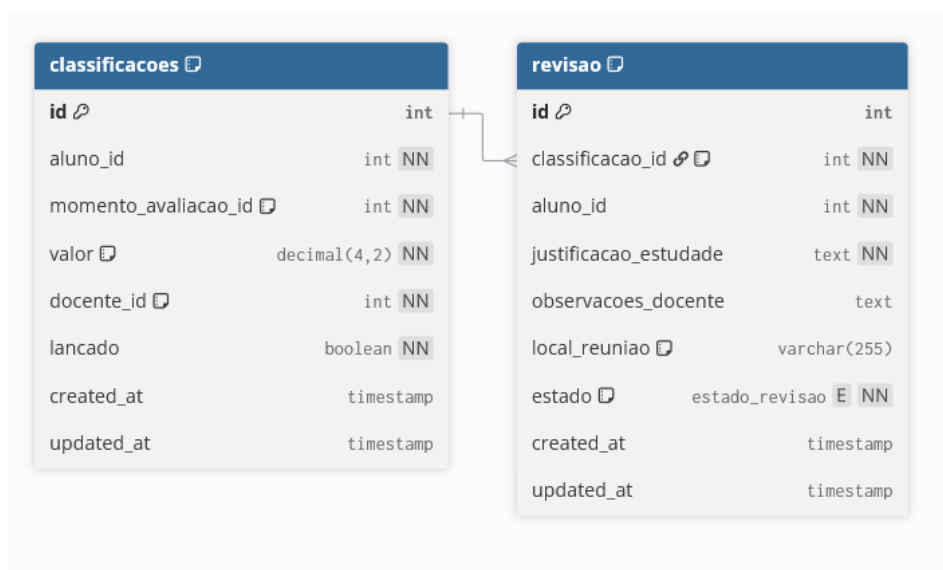


FIGURA 24 - BASE DE DADOS DO MICROSERVIÇO DE CLASSIFICAÇÕES

Configuração de Avaliações

Esta representação demonstra a base de dados da entidade “Configuração de Avaliações”, onde é possível descrever os detalhes de avaliação de uma dada UC, por exemplo, é possível descrever que a avaliação das componentes de uma UC são 70% avaliação prática e 30% avaliação teórica.

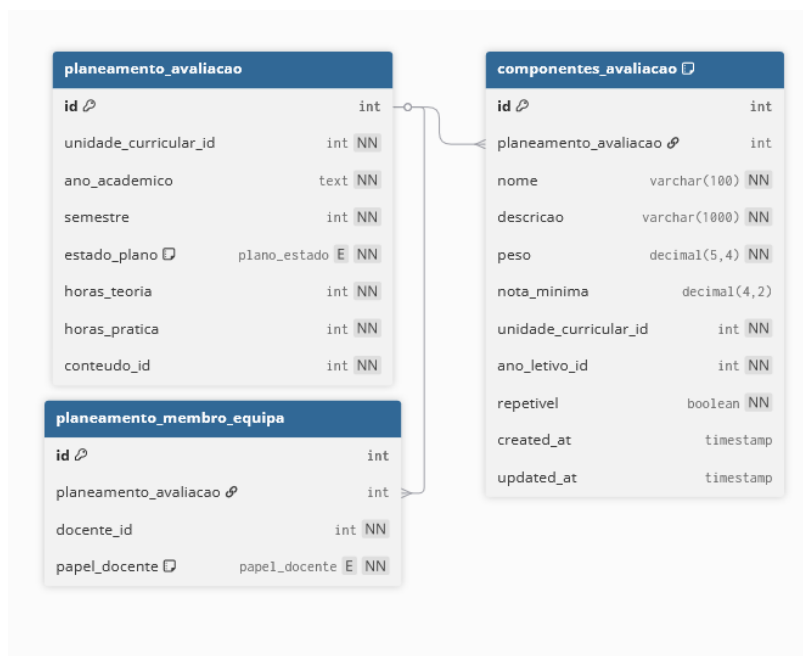


FIGURA 25 - BASE DE DADOS DO MICROSERVIÇO DE CONFIGURAÇÃO DE AVALIAÇÕES

Pautas

Esta representação demonstra a base de dados da entidade “Pautas”, onde demonstra o ato de publicar as classificações de uma UC, até ao ponto final onde o docente assina essa pauta, tornando-a permanente.

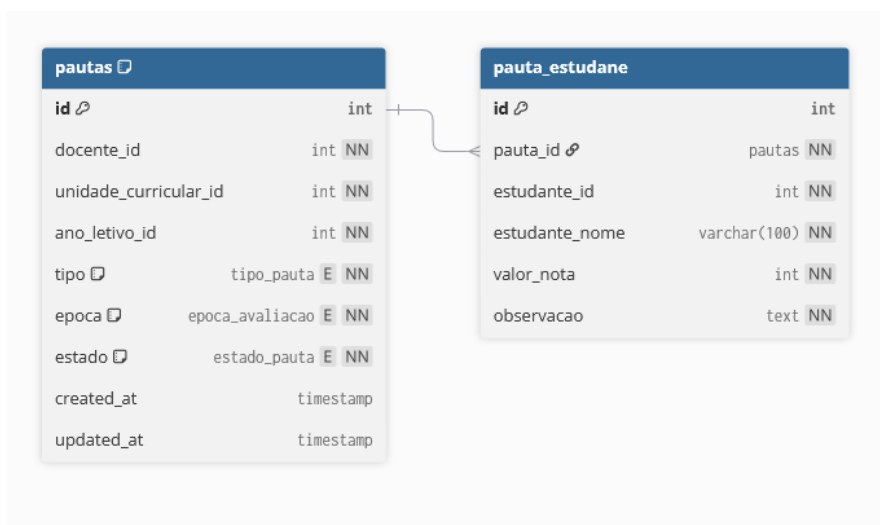


FIGURA 26 - BASE DE DADOS DO MICROSERVIÇO DE PAUTAS

Candidaturas

Esta representação demonstra a base de dados da entidade “Candidaturas”. Esta entidade representa a candidatura de um indivíduo a uma instituição de ensino superior, com o upload de documentos necessários para a efetuar, a fase em que está a concorrer, e as suas opções de curso.

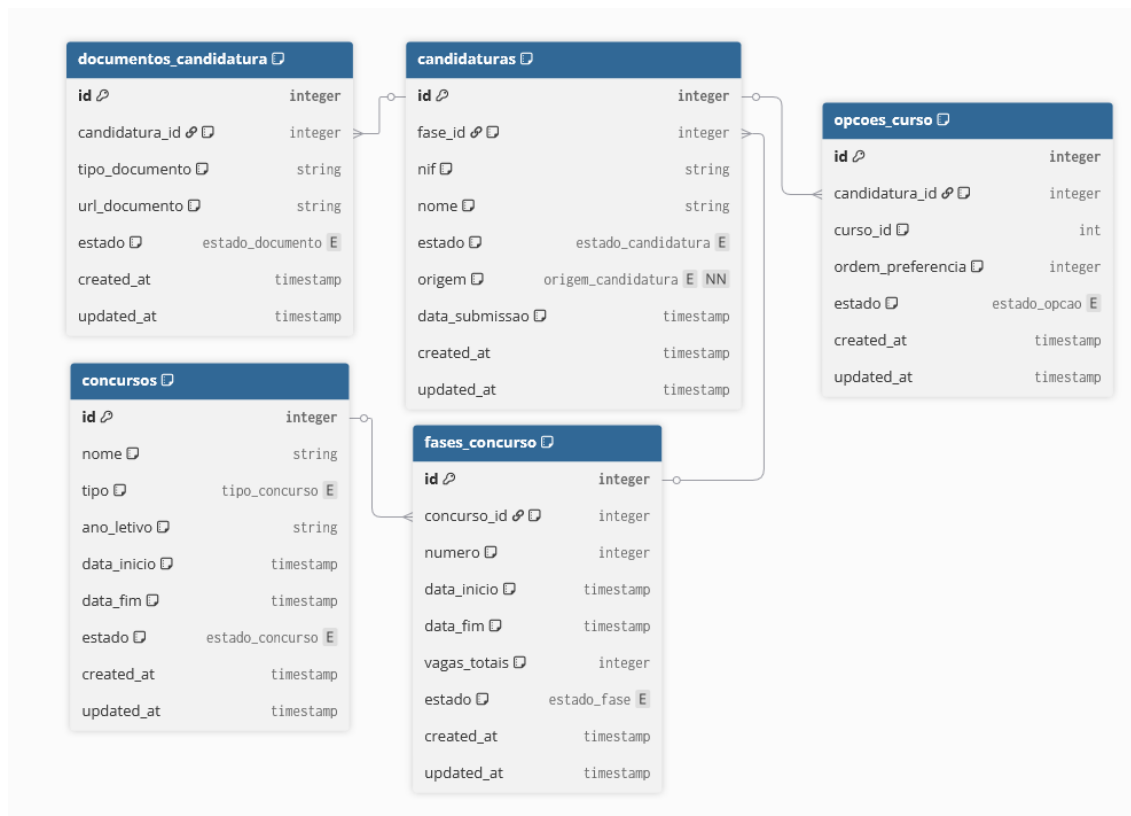


FIGURA 27 - BASE DE DADOS DO MICROSERVIÇO DE CANDIDATURAS

Matrículas

Esta representação demonstra a base de dados da entidade “Matrículas”. Após a candidatura de um indivíduo a uma instituição de ensino superior, esta entidade representa a matrícula desse aluno nas várias UCs do curso a que concorreu, com upload de documentos para a oficialização dessa matrícula.

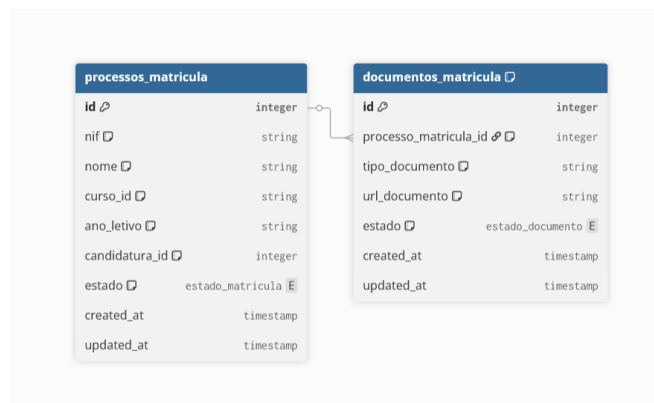


FIGURA 28 - BASE DE DADOS DO MICROSERVIÇO DE MATRÍCULAS

Gestão Académica

Esta representação demonstra a base de dados da entidade “Gestão Académica”, onde é possível gerir todo o ecossistema educacional, nomeadamente o ciclo completo de informação relacionado com a organização de turmas, aulas, disciplinas, cursos, bem como os registos de professores e alunos.



FIGURA 29 - BASE DE DADOS DO MICROSERVIÇO DE GESTÃO ACADÉMICA

Horários

Esta representação demonstra a base de dados da entidade “Horários”, onde é possível gerir o espaço físico das aulas (Sala), o seu horário de disponibilidade, marcações e exceções (horários em que a sala deveria estar disponível, mas houve uma marcação extra)

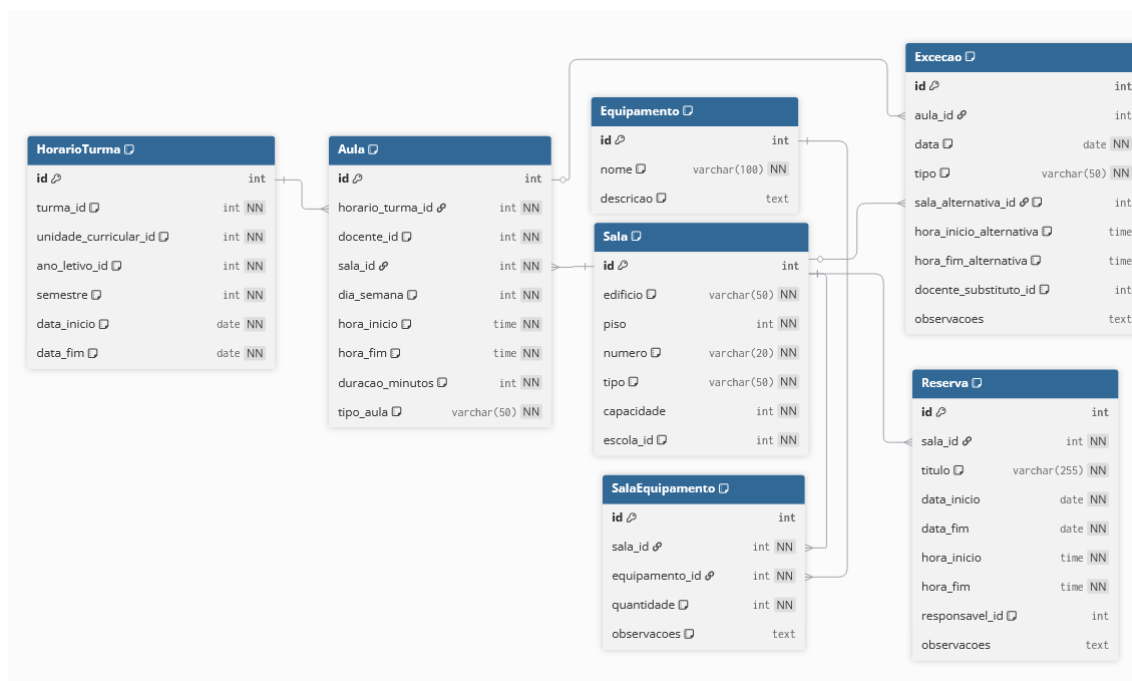


FIGURA 30 - BASE DE DADOS DO MICROSERVIÇO DE HORÁRIOS

IAM

Esta representação demonstra o modelo de dados para o subsistema de Identificação e Gestão de Acessos (IAM). O diagrama detalha a estrutura de Autenticação e Autorização, estabelecendo a gestão de Utilizadores e definindo as relações entre estes e os papéis que lhes são atribuídos, bem como as permissões específicas associadas a cada papel.

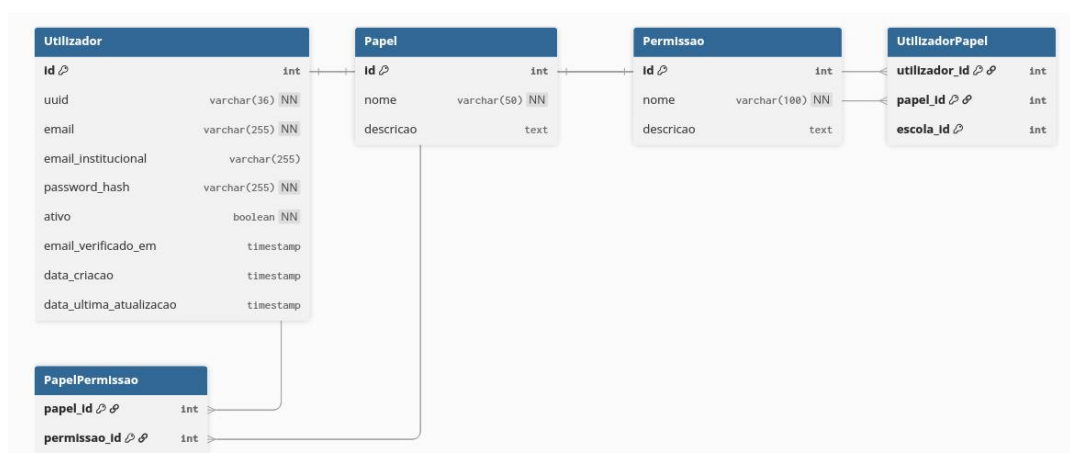


FIGURA 31 - BASE DE DADOS DO MICROSERVIÇO DE IAM

Notificações

Esta representação demonstra a entidade Notificação, onde será possível enviar e receber mensagens da plataforma ou emails.



Notificacao		
id	int	
utilizador_id	long	NN
email	varchar(25)	NN
titulo	varchar(100)	NN
message	text	NN
canal	varchar(20)	NN
estado	varchar(50)	NN
erro	text	
data_criacao	timestamp	
data_envio	timestamp	

FIGURA 32 - BASE DE DADOS DO MICROSERVIÇO DE NOTIFICAÇÕES

10. Conclusão

10.1. Reflexão Crítica dos Resultado

O desenvolvimento deste projeto permitiu-nos aplicar na prática os conceitos teóricos abordados na unidade curricular, nomeadamente a arquitetura de microserviços e os princípios do Domain-Driven Design (DDD). Apesar dos constrangimentos de tempo, conseguimos especificar, desenhar e implementar a maior parte dos microserviços planeados para o Sistema Integrado de Gestão Académica (SIGA).

A abordagem híbrida de metodologias (SCRUM e Waterfall) revelou-se adequada, permitindo um planeamento detalhado inicial e uma implementação iterativa e colaborativa. A utilização do Docker e Kubernetes (Minikube) facilitou a orquestração e o deploy dos serviços, enquanto a integração do RabbitMQ permitiu implementar comunicação assíncrona robusta entre microserviços, como entre Candidaturas e Matrículas.

Um dos principais desafios foi a gestão da consistência de dados distribuídos e a definição clara dos contextos limitados. A falta de tempo impediu a implementação completa do microserviço de IAM (Identidade e Acesso), que era fundamental para a segurança e controlo de acesso da plataforma. Esta limitação constitui uma falha significativa na versão atual, uma vez que a autenticação e autorização são pilares essenciais num sistema deste tipo.

Globalmente, o sistema demonstra a viabilidade da abordagem de microserviços para domínios complexos como a gestão académica, permitindo escalabilidade, independência de tecnologias por serviço e um melhor alinhamento entre a equipa de desenvolvimento e o domínio de negócio.

10.2. GitLab e Jira

Devido à utilização da versão gratuita do GitLab, apenas foi possível adicionar um dos docentes da unidade curricular, dado que o limite de intervenientes por projeto é de 5 utilizadores.

<https://gitlab.com/mtds2/mtds>

A gestão do projeto e a divisão de trabalho foram realizadas através da plataforma Jira, utilizando o conceito de Issues (tarefas). Cada Issue representou uma funcionalidade específica ou uma tarefa individual, sendo posteriormente atribuída a um membro da equipa. A complexidade e o esforço estimado de cada Issue foram quantificados através de Story Points, permitindo uma distribuição de carga de trabalho equilibrada entre os intervenientes

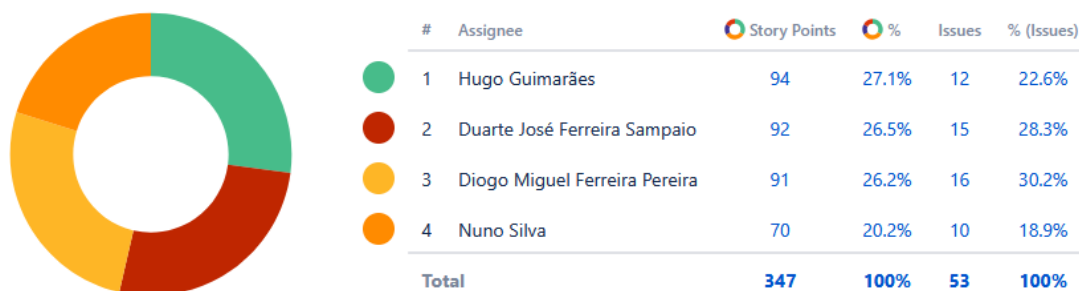


FIGURA 33 - GRÁFICO DE DEMONSTRAÇÃO DE DIVISÃO DE TRABALHO

10.3. Conclusões e Trabalho Futuro

Como trabalho futuro, a prioridade será a implementação do microserviço de IAM, garantindo mecanismos robustos de autenticação (JWT/OAuth2) e gestão de papéis e permissões. Adicionalmente, planeia-se:

- **Colocar o sistema online (ex.: AWS ou Azure):** configurar o cluster *Kubernetes* e a API Gateway para acesso público;
- **Completar e reforçar os testes de integração entre todos os microserviços,** especialmente os que envolvem comunicação síncrona e assíncrona.

Este projeto serviu como uma prova de conceito valiosa, confirmando o potencial da arquitetura, mas também evidenciando a complexidade operacional e a necessidade de uma gestão cuidada da consistência, segurança e, sobretudo, do *deploy* e operação em ambiente real.

11. Anexos

Devido à utilização da versão gratuita do GitLab, apenas foi possível adicionar um dos docentes da unidade curricular, dado que o limite de intervenientes por projeto é de 5 utilizadores.

<https://gitlab.com/mtds2/mtds>



#	Assignee	Story Points	%	Issues	% (Issues)
1	Hugo Guimarães	94	27.1%	12	22.6%
2	Duarte José Ferreira Sampaio	92	26.5%	15	28.3%
3	Diogo Miguel Ferreira Pereira	91	26.2%	16	30.2%
4	Nuno Silva	70	20.2%	10	18.9%
Total		347	100%	53	100%

Referências

- [1] Ricardo Santos, "Slides Domain-Driven Design." Accessed: Dec. 02, 2025. [Online]. Available:
https://moodle.estg.ipp.pt/pluginfile.php/12275/mod_resource/content/0/2025_MTS_DS_Aula03.pdf
- [2] "Domain-Driven Design (DDD): Um Resumo – Engenharia de Software Moderna." Accessed: Dec. 01, 2025. [Online]. Available:
<https://engsoftmoderna.info/artigos/ddd.html>
- [3] "GitHub - ernesen/DDD: Implementing Domain-Driven Design for Microservice Architecture." Accessed: Dec. 01, 2025. [Online]. Available:
<https://github.com/ernesen/DDD>
- [4] K. Wagh and R. Thool, "A Comparative Study of SOAP Vs REST Web Services Provisioning Techniques for Mobile Host," *Journal of Information Engineering and Applications*, vol. 2, 2012, [Online]. Available: www.iiste.org